

P390

Britt Anderson

2025-01-01

Table of contents

Preface	1
1 Introduction	3
2 How can a computer help us in our research?	5
2.1 Open Research Practices	5
3 Basic Tools for Scientific Computing	7
4 Integrated Development Environments (IDEs)	9
4.1 Required for this Course: VS Code	9
4.1.1 Installing VS Code	9
4.1.2 Using VSCode	9
4.2 Other Options	9
5 Terminals	11
5.1 Commands to try (many have options)	11
5.2 Additional Introductory Material	12
5.3 Getting work done: scripting	12
5.4 Talking to other computers in the terminal	12
5.4.1 SSH	12
6 Terminal Self-Learning Exercises for Psychology Students	13
6.1 Prerequisites	13
6.2 Note on Command Compatibility	13
6.3 Exercise Set 1: Orientation and Basic Navigation	13
6.3.1 Learning Goals	13
6.3.2 1.1 Where Am I?	14
6.3.3 1.2 What's Here?	14
6.3.4 1.3 More Details	14
6.3.5 1.4 Including Hidden Files	14
6.3.6 1.5 Moving Around	15
6.3.7 1.6 Going Back	15
6.3.8 1.7 Returning Home	15

6.4	Exercise Set 2: Creating and Manipulating Files	15
6.4.1	Learning Goals	15
6.4.2	2.1 Make a Workspace	16
6.4.3	2.2 Create a File	16
6.4.4	2.3 Add Content to a File	16
6.4.5	2.4 Append to a File	16
6.4.6	2.5 Copy Files	17
6.4.7	2.6 Rename/Move Files	17
6.4.8	2.7 Create Directory Structure	17
6.4.9	2.8 Move Files Between Directories	17
6.4.10	2.9 Safe Deletion	17
6.4.11	2.10 Remove Directories	18
6.5	Exercise Set 3: Viewing and Searching Content	18
6.5.1	Learning Goals	18
6.5.2	3.1 Setup the Dataset	18
6.5.3	3.2 Page Through Files	19
6.5.4	3.3 First Few Lines	19
6.5.5	3.4 Last Few Lines	19
6.5.6	3.5 Count Lines	19
6.5.7	3.6 Search for Patterns	20
6.5.8	3.7 Search with Count	20
6.5.9	3.8 Case-Insensitive Search	20
6.5.10	3.9 Combining Commands (Pipes)	20
6.5.11	3.10 Sorting Data	20
6.6	Exercise Set 4: Efficiency Tools	21
6.6.1	Learning Goals	21
6.6.2	4.1 Command History	21
6.6.3	4.2 Search History	21
6.6.4	4.3 Tab Completion	21
6.6.5	4.4 Cursor Movement	22
6.6.6	4.5 Wildcards - Multiple Files	22
6.6.7	4.6 Wildcards - Single Character	22
6.6.8	4.7 Clear the Screen	22
6.7	Exercise Set 5: Practical Research Workflow	23
6.7.1	Learning Goals	23
6.7.2	5.1 Organize a Project	23
6.7.3	5.2 Create a Data Log	23
6.7.4	5.3 Document Your Commands	24
6.7.5	5.4 Make Scripts Executable	24
6.7.6	5.5 Batch Process Files	24
6.7.7	5.6 Extract Specific Data	25
6.7.8	5.7 Create Summary Statistics	25
6.7.9	5.8 Find Files by Content	25
6.7.10	5.9 Check Disk Usage	26
6.7.11	5.10 Final Cleanup Exercise	26
6.8	Self-Assessment Checklist	26

6.9	Common Pitfalls and Tips	27
6.9.1	Safety	27
6.9.2	Efficiency	27
6.9.3	Debugging	27
6.9.4	Getting Help	27
6.10	Next Steps	27
6.11	Getting Stuck?	28
7	Version Control	29
7.1	Git	29
7.2	An old overview of git and Github	29
7.2.1	Getting Git	29
7.2.2	Configuring Git	29
7.3	Other version control system	29
7.4	Git is not Github	30
7.5	Using git with github	30
7.5.1	Creating an ssh key with your terminal	30
7.6	Understanding Git and the Workflow in Pictures	30
7.7	Principal Terms Terms for Git	30
8	Lab Notebooks	33
9	Writing Scientific Documents	35
9.1	IDEs and Writing up your Research	35
9.1.1	VSCode	35
9.1.2	Some helpful links to get you started	36
9.1.3	RStudio	36
9.2	Scientific Publishing Tools	36
9.3	Things You Can Do With Quarto	37
9.3.1	Making a Website with Quarto and Github Pages	37
9.3.2	Journal Article Authoring	40
9.3.3	Presentations	40
10	Coding General	43
10.1	Languages	43
10.2	Common Programming Constructs	43
10.2.1	Variables	43
10.2.2	Loops	44
10.2.3	Virtual Environments	44
10.2.4	Assignment and Equality Are Different in Programming	45
11	R	47
11.1	Installation	47
11.1.1	Test your installation	47
11.2	Interfacing with VS Code	47
11.2.1	Resources	47

11.3	Installing R Packages	48
11.4	Mix R Code with Text for Reproducible and Documented Workflows	48
11.5	R Types	49
11.6	Data.Frames	50
11.7	Selecting Data	50
11.8	Practice	51
11.9	Looping in R	51
11.10	More Practice	52
11.11	Writing a Function in R	52
12	Python	53
12.1	Installation	53
12.2	Packages	53
12.2.1	Using uv to create an env and install local packages . . .	53
12.2.2	Testing Your Python Package Installation	54
12.3	Loops in Python	55
12.3.1	Loops in Python Make Use of Iterables	56
12.3.2	Additional Examples of Programming Constructs in Python	56
12.4	Popular and Useful Python Libraries	58
13	Programming Experiments	59
13.1	All-in-one options	59
13.2	Python	60
13.2.1	Pygame	60
13.3	Javascript	62
14	In Lab Experiments	63
14.0.1	Online Experiments	63
15	Handling Data	65
16	Knowledge Management	71
16.1	(Zettlekasten)	71
16.2	Reference Management	73
16.2.1	Finding Citations	73
16.2.2	More on Citation Management	73
16.2.3	The last thing you want to do	74
16.3	Using References With Quatro and VS Code	74
16.4	Using References With Overleaf	75
17	Testing Your Javascript	77
17.1	Run your own server locally	77
17.1.1	XAMPP	77
17.1.2	In Lab Experiments	77
17.1.3	Online Experiments	77
18	Large Language Models	79

18.1 Run locally	79
18.2 How to?	79
18.2.1 Key Features of GPT4ALL	79
18.3 How-to	80
18.4 Using LLMs for Dynamic Research	80
18.5 Why Might You Want To Do This?	81
18.6 Next steps	81
19 Databases and management	83
19.0.1 Datascience	83
19.0.2 Grant Funding	83
20 Summary	85
Appendices	87
A Mac Related Instructions	87
A.1 Homebrew	87
A.2 Git	87
A.3 VS Code	87
A.4 Texlive	87
A.5 UV	87
A.6 R	88
A.7 Quarto	88
B Windows Specific Advice	89
B.1 Windows Subsystem for Linux	89
B.1.1 Make sure you are on Windows 10 or 11 and that you are updated.	89
B.1.2 Then follow the instructions per microsoft	89
B.2 VSCode	89
B.3 Git	89
B.4 Python	89
C Python	91
D Virtual Environments	93
D.1 Methods	93
D.1.1 Creating and Activating the Venv	93
D.1.2 The Terminal is All You Need	94
D.2 Visual Studio Code	94
D.2.1 Quarto Complications	95

Preface

This course is an effort to expose research minded undergraduate students in psychology to a variety of computational and programming tools that they can use to ease their research tasks. It is also intended to provide them practice at using such tools. Although I hope the particular tools that I choose for this are useful now, they may not be in the future. Things change fast in this area: smartphones are less than twenty years old, and practically useful chatbots are less than five years old. The emphasis is therefore on learning how to teach yourself to learn to use useful computer based tools.

These notes are written as a series of `qmd` files. This makes it a Quarto book. Quarto can be seen as a successor to RStudio and `Rmd` files. But rather than focusing on Quarto or `qmd` files per se, what I hope people will take away is that plain text files are enduring, human readable, and easily shareable. Computers can help with all the complicated syntax, and that is where **markdown** variants are useful. `qmd` files are one such variant, and Quarto is the tool that takes our plain text and turns it into something pretty. First, we wrote in `notepad`, then we used RStudio, and now we are using Quarto (which you can use in RStudio, but which we will use from inside VSCode). What will you do when Quarto is passé? Or a collaborator wants to use a different markdown variant? You need to be able to understand how to change your syntax, download and install the new tool. That meta-knowledge should be your real goal in this course.

The subsequent material is generally ordered in the way I intend to teach it. These notes are not self-contained, but are intended to be reference material. If I mention a tool in class this is where you can find the name and link if you did not catch it. I may also demonstrate tool use in these pages and there will often be code that you can cut and paste to try things out. Think of this book more as a “student’s guide” than as a textbook.

Chapter 1

Introduction

This is a book being created for PSYCH390 at the the University of Waterloo.

It is very much a work in progress. You can help it get better by providing feedback. Ideally you will do this by raising an issue on the github repo. Even better is if you would fix the book yourself and make a pull request.

Chapter 2

How can a computer help us in our research?

Class Question

What are steps (or stages) involved in psychological research?

2.1 Open Research Practices

Class Question

- What is “open research”?
- Should research be open?

10 strategies for open science

Class Question

What are the **Fair** principles for research?

- Fair principles
- Fair guiding principles

Read and discuss: A toolkit for transparency

Chapter 3

Basic Tools for Scientific Computing

In the following some of the common tools are highlighted with instructions for how to download and install.

Chapter 4

Integrated Development Environments (IDEs)

- Who are you writing code for? Human or Machine?
- What is an IDE? What makes for a good IDE?

4.1 Required for this Course: VS Code

VSCode Opensource alternative

4.1.1 Installing VS Code

See the operating system specific appendices.

4.1.2 Using VSCode

<https://www.youtube.com/watch?v=B-s71n0dHUk>

- Basics video
- Using VSCode with R
- Using VSCode with Python

4.2 Other Options

Emacs. VIM Jupyter Notebooks An environment for interactively analyzing and visualizing data. Can generate very nice output. It can be mixed with VS Code, but is most often used directly in a web browser. <https://www.youtube.com/watch?v=suAkMeWJ1yE>

Chapter 5

Terminals

Currently these words are used as synonyms: terminal, shell, command line; which is not exactly true, but is close enough.

<https://vimeo.com/453837142>

- Power Shell.
- OSX Terminal

5.1 Commands to try (many have options)

- `ls`
- `ls -alt`
- `cd`
- `pwd`
- `mkdir`
- `rm` **DANGER**
- `mv`
- `cp`
- `cat`
- `less`
- `find` **very powerful**
- `du`
- `df`
- `touch`
- `ln`
- `chmod`
- `chown`

5.2 Additional Introductory Material

1. The command line
2. A Short Series of Terminal Lessons from the UbuntuWiki
3. Some Scripting Basics
4. Another Scripting Introduction

5.3 Getting work done: scripting

Want to convert and compress a large directory of videos (as I did for the pandemic version of a course)?

```
for i in *.MP4;
do name=`echo "$i" | cut -d'.' -f1`;
echo "$name";
ffmpeg -i "$i" -c:v libx264 -b:v 1.5M -c:a aac -b:a 128k "${name}S.mp4";
done
```

5.4 Talking to other computers in the terminal

5.4.1 SSH

Testing and Learning for Linux SSH

Chapter 6

Terminal Self-Learning Exercises for Psychology Students

6.1 Prerequisites

- You have located your terminal application (Terminal on Mac, WSL terminal on Windows)
- You can open it successfully

6.2 Note on Command Compatibility

These exercises work in both bash (WSL/Linux) and zsh (Mac). The commands are identical between these shells.

6.3 Exercise Set 1: Orientation and Basic Navigation

6.3.1 Learning Goals

- Understand what the prompt means
- Know where you are in the file system
- Navigate between directories

6.3.2 1.1 Where Am I?

Open your terminal. You'll see a prompt (something like `user@computer:~$` or `computer-name:~ user$`).

Task: Type `pwd` and press Enter.

Question to answer: What does `pwd` stand for? What information does it give you?

Expected outcome

You should see a path like `/home/username` (Linux/WSL) or `/Users/username` (Mac). This is your “home directory” - your starting location.

`pwd` = “print working directory”

6.3.3 1.2 What's Here?

Task: Type `ls` and press Enter.

Question to answer: What does this command show you?

Expected outcome

You see a list of files and folders in your current directory. Common ones might include Documents, Downloads, Desktop.

`ls` = “list”

6.3.4 1.3 More Details

Task: Type `ls -l` and press Enter.

Question to answer: How is the output different from plain `ls`? What additional information do you see?

Expected outcome

You see the same files but with permissions, owner, size, and modification date. The `-l` is called a “flag” or “option” that modifies command behavior.

6.3.5 1.4 Including Hidden Files

Task: Type `ls -la` and press Enter.

Question to answer: Do you see any files starting with `.` (period)? What do you think these are?

Expected outcome

Files starting with `.` are hidden by default. They're often configuration files like `.bashrc` or `.zshrc`.

6.3.6 1.5 Moving Around

Task: 1. Type `cd Documents` and press Enter 2. Type `pwd` to see where you are now 3. Type `ls` to see what's in Documents

Question to answer: Did your prompt change? How can you tell you're in a different location?

Expected outcome

Your working directory changed to `/home/username/Documents` (or similar). The prompt might show the directory name.

`cd` = “change directory”

6.3.7 1.6 Going Back

Task: Type `cd ..` and press Enter, then `pwd`.

Question to answer: Where are you now? What does `..` mean?

Expected outcome

You're back in your home directory. `..` means “parent directory” (one level up).

6.3.8 1.7 Returning Home

Task: 1. Navigate into Documents again: `cd Documents` 2. Type `cd` (with nothing after it) and press Enter 3. Type `pwd` to verify

Question to answer: Where does `cd` by itself take you?

Expected outcome

Plain `cd` always returns you to your home directory, no matter where you are.

6.4 Exercise Set 2: Creating and Manipulating Files

6.4.1 Learning Goals

- Create directories and files
- Move and copy things
- Delete safely

6.4.2 2.1 Make a Workspace

Task: 1. Make sure you're in your home directory: `cd` 2. Type `mkdir terminal_practice` and press Enter 3. Type `ls` to verify it was created 4. Type `cd terminal_practice`

Question to answer: What do you think `mkdir` stands for?

Expected outcome

You created a new directory and moved into it.

`mkdir` = "make directory"

6.4.3 2.2 Create a File

Task: Type `touch experiment_notes.txt` and press Enter, then `ls`.

Question to answer: Does a file appear? Open your graphical file browser (Finder/Explorer) and navigate to this folder. Can you see the file there?

Expected outcome

You created an empty file. `touch` creates files (technically, it updates timestamps, but creates the file if it doesn't exist).

6.4.4 2.3 Add Content to a File

Task: Type `echo "First observation: Subjects prefer terminal over GUI" > experiment_notes.txt` and press Enter.

Question to answer: What do you think `>` does? Try opening the file in a text editor to check.

Expected outcome

The text was written to the file. `>` means "redirect output to file" (it overwrites existing content).

6.4.5 2.4 Append to a File

Task: 1. Type `echo "Second observation: Learning curve steep" >> experiment_notes.txt` 2. View the file content: `cat experiment_notes.txt`

Question to answer: How is `>>` different from `>`?

Expected outcome

You see both lines. `>>` appends (adds to the end) instead of overwriting.

`cat` = "concatenate" (displays file contents)

6.4.6 2.5 Copy Files

Task: 1. Type `cp experiment_notes.txt backup_notes.txt` 2. Type `ls` to verify both files exist

Question to answer: What do you think `cp` stands for?

Expected outcome

You now have two identical files.

`cp` = “copy” Format: `cp source destination`

6.4.7 2.6 Rename/Move Files

Task: Type `mv backup_notes.txt observations.txt` and then `ls`.

Question to answer: What happened to `backup_notes.txt`?

Expected outcome

The file was renamed. `mv` = “move” but is also used for renaming (moving to a new name in the same directory).

6.4.8 2.7 Create Directory Structure

Task: 1. Type `mkdir data` 2. Type `mkdir data/raw data/processed` 3. Type `ls data`

Question to answer: Can you create multiple directories at once?

Expected outcome

You created a `data` directory with two subdirectories. You can specify multiple paths to `mkdir`.

6.4.9 2.8 Move Files Between Directories

Task: 1. Create a test file: `touch subject_001_data.csv` 2. Move it: `mv subject_001_data.csv data/raw/` 3. Verify: `ls data/raw`

Question to answer: How do you move files between directories?

Expected outcome

The file is now in `data/raw/`. When moving, you specify the destination directory.

6.4.10 2.9 Safe Deletion

Task: 1. Type `rm observations.txt` 2. Type `ls` to verify it’s gone 3. Try `ls observations.txt`

Question to answer: Can you get the file back after using `rm`?

Expected outcome

The file is permanently deleted. There's no "recycle bin" in the terminal. **Be careful with rm!**

`rm` = "remove"

6.4.11 2.10 Remove Directories

Task: 1. Try `rm data` 2. Now try `rm -r data`

Question to answer: Why doesn't the first command work? What does the `-r` flag do?

Expected outcome

Plain `rm` doesn't work on directories. The `-r` flag means "recursive" (remove directory and everything inside).

Warning: `rm -r` is dangerous - it deletes everything in the directory. Always double-check before using it.

6.5 Exercise Set 3: Viewing and Searching Content

6.5.1 Learning Goals

- Read file contents in different ways
- Search within files
- Understand pipes and filters

6.5.2 3.1 Setup the Dataset

Task: Let's create a sample dataset for practice.

```
cd ~/terminal_practice
cat > participants.txt << 'EOF'
ID,Age,Condition,Score
001,23,Control,78
002,27,Treatment,85
003,21,Control,72
004,25,Treatment,91
005,29,Control,68
006,22,Treatment,88
007,26,Control,75
008,24,Treatment,82
EOF
```

Just copy all of this and paste it into your terminal, then press Enter.

Question to answer: Type `cat participants.txt` - do you see your data?

Expected outcome

You created a CSV file with participant data. The `<< 'EOF'` syntax is called a “here document” - a way to input multiple lines.

6.5.3 3.2 Page Through Files

Task: 1. Type `less participants.txt` and press Enter 2. Use arrow keys to navigate 3. Press `q` to quit

Question to answer: How is `less` different from `cat`?

Expected outcome

`less` allows you to scroll through files without printing everything to the screen. Useful for large files.

Navigation: arrows to move, `q` to quit, `/` to search

6.5.4 3.3 First Few Lines

Task: Type `head participants.txt`

Question to answer: How many lines does it show?

Expected outcome

Shows first 10 lines by default. Useful for checking file structure.

Try: `head -n 3 participants.txt` to show only 3 lines

6.5.5 3.4 Last Few Lines

Task: Type `tail participants.txt`

Question to answer: What do you see?

Expected outcome

Shows last 10 lines. Useful for checking end of files or recent log entries.

6.5.6 3.5 Count Lines

Task: Type `wc -l participants.txt`

Question to answer: What number do you get? Why?

Expected outcome

You should see 8 `participants.txt` (or possibly 9 depending on blank lines).

`wc` = “word count”, `-l` flag means count lines instead

6.5.7 3.6 Search for Patterns

Task: Type `grep Treatment participants.txt`

Question to answer: What gets displayed?

Expected outcome

Only lines containing “Treatment” are shown. `grep` searches for patterns in files.

6.5.8 3.7 Search with Count

Task: Type `grep -c Treatment participants.txt`

Question to answer: What does this number represent?

Expected outcome

The count of lines containing “Treatment”. The `-c` flag gives you a count instead of the lines themselves.

6.5.9 3.8 Case-Insensitive Search

Task: 1. Type `grep control participants.txt` 2. Type `grep -i control participants.txt`

Question to answer: Why different results?

Expected outcome

First search finds nothing (lowercase doesn’t match “Control”). Second search with `-i` flag is case-insensitive.

6.5.10 3.9 Combining Commands (Pipes)

Task: Type `cat participants.txt | grep Treatment | wc -l`

Question to answer: What happened? What does the `|` symbol do?

Expected outcome

This counts Treatment participants. The `|` (pipe) sends output of one command as input to the next.

Flow: display file → filter for Treatment → count lines

6.5.11 3.10 Sorting Data

Task: 1. Type `sort participants.txt` 2. Type `sort -k4 -t, -n participants.txt`

Question to answer: How does the second command change the sort?

Expected outcome

First sorts alphabetically by whole line. Second sorts numerically (**-n**) by field 4 (**-k4**) using comma as delimiter (**-t,**) - i.e., by Score column.

6.6 Exercise Set 4: Efficiency Tools

6.6.1 Learning Goals

- Use command history
- Autocomplete with Tab
- Edit commands efficiently
- Use wildcards

6.6.2 4.1 Command History

Task: 1. Type **history** and press Enter 2. Use up-arrow key a few times 3. Press down-arrow

Question to answer: What do the arrow keys do?

Expected outcome

history shows all past commands. Up/down arrows cycle through command history - very useful for repeating or modifying previous commands.

6.6.3 4.2 Search History

Task: 1. Press **Ctrl+r** (Control-r) 2. Start typing **grep** 3. Press **Ctrl+r** again to see next match

Question to answer: What does **Ctrl+r** do?

Expected outcome

Reverse search through command history. Start typing and it finds matching commands. Press Enter to run, or edit first.

6.6.4 4.3 Tab Completion

Task: 1. Type **cd ~/term** and press Tab 2. If it doesn't complete, press Tab twice

Question to answer: What happened?

Expected outcome

Tab autocompletes file/directory names. Double-tab shows all matches if there are multiple options.

This saves enormous amounts of typing and prevents typos.

6.6.5 4.4 Cursor Movement

Task: Type a long command (don't press Enter yet): `echo "This is a very long command that I want to edit"`

Now practice: - `Ctrl+a` (jump to start) - `Ctrl+e` (jump to end) - `Alt+b` or `Esc` then `b` (back one word) - `Alt+f` or `Esc` then `f` (forward one word)

Question to answer: Are these faster than arrow keys?

Expected outcome

These shortcuts make editing commands much faster than holding arrow keys.

6.6.6 4.5 Wildcards - Multiple Files

Task: 1. Create test files: `touch data_jan.csv data_feb.csv data_mar.csv report_jan.txt` 2. Type `ls data*` 3. Type `ls *.csv` 4. Type `ls data_*.csv`

Question to answer: What does `*` match?

Expected outcome

`*` is a wildcard matching any characters. - `data*` matches anything starting with "data" - `*.csv` matches anything ending with ".csv"
- `data_*.csv` matches that pattern

This is powerful for operating on multiple files at once.

6.6.7 4.6 Wildcards - Single Character

Task: 1. Type `ls data_???.csv`

Question to answer: How is `?` different from `*`?

Expected outcome

`?` matches exactly one character. So `???` matches exactly three characters (jan, feb, mar).

6.6.8 4.7 Clear the Screen

Task: 1. Your terminal is probably cluttered now. Type `clear` 2. Alternatively, press `Ctrl+l`

Question to answer: Does this delete your history?

Expected outcome

Screen is cleared but history remains. You can still scroll up or use up-arrow to access previous commands.

6.7 Exercise Set 5: Practical Research Workflow

6.7.1 Learning Goals

- Combine tools for real tasks
- Develop a reproducible workflow
- Practice documentation

6.7.2 5.1 Organize a Project

Task: Create this directory structure:

```
cd ~
mkdir psych_study
cd psych_study
mkdir data data/raw data/processed scripts results
```

Then verify: `ls -R`

Question to answer: What does `ls -R` do?

Expected outcome

You created a standard research project structure. `-R` shows recursive listing (shows subdirectories too).

6.7.3 5.2 Create a Data Log

Task:

```
cd ~/psych_study
echo "# Data Collection Log" > README.txt
echo "" >> README.txt
echo "Date: $(date)" >> README.txt
echo "Researcher: [Your Name]" >> README.txt
echo "Study: Terminal Skills Acquisition" >> README.txt
```

Then view it: `cat README.txt`

Question to answer: What does `$(date)` do?

Expected outcome

`$(date)` runs the date command and inserts its output. This is called “command substitution” - very useful in scripts.

6.7.4 5.3 Document Your Commands

Task: Create a script file:

```
cd ~/psych_study/scripts
touch process_data.sh
echo "#!/bin/bash" > process_data.sh
echo "# Script to process participant data" >> process_data.sh
echo "" >> process_data.sh
echo "echo 'Starting data processing...'" >> process_data.sh
echo "ls ../data/raw/" >> process_data.sh
echo "echo 'Processing complete!'" >> process_data.sh
```

View it: `cat process_data.sh`

Question to answer: What is `#!/bin/bash` at the top?

Expected outcome

This is called a “shebang” - it tells the system this is a bash script. Scripts are just text files containing commands.

6.7.5 5.4 Make Scripts Executable

Task: 1. Try running: `./process_data.sh` 2. It fails! Now type: `chmod +x process_data.sh` 3. Try again: `./process_data.sh`

Question to answer: What did `chmod +x` do?

Expected outcome

`chmod +x` makes the file executable. Without it, the system won’t run it.

`chmod` = “change mode” (permissions)

6.7.6 5.5 Batch Process Files

Task: Create sample data files and process them:

```
cd ~/psych_study/data/raw
echo "subject,rt,accuracy" > pilot_01.csv
echo "S001,450,1" >> pilot_01.csv
echo "S002,523,1" >> pilot_01.csv

echo "subject,rt,accuracy" > pilot_02.csv
echo "S003,489,0" >> pilot_02.csv
echo "S004,512,1" >> pilot_02.csv

# Now count how many data rows across all files
cat pilot_*.csv | grep -v "subject" | wc -l
```


Question to answer: What does `grep -v` do?

Expected outcome

You counted total participants across multiple files. `grep -v` inverts the match (shows lines NOT containing the pattern) - here, excluding headers.

6.7.7 5.6 Extract Specific Data

Task:

```
# Extract all accurate responses (accuracy=1)
grep ",1$" pilot_*.csv
```

Question to answer: What does `$` mean in the pattern?

Expected outcome

`$` means “end of line” in `grep` patterns. So `,1$` matches lines ending with “,1”.

This is a “regular expression” - a powerful pattern matching system.

6.7.8 5.7 Create Summary Statistics

Task:

```
cd ~/psych_study/results
echo "# Summary Statistics" > summary.txt
echo "" >> summary.txt
echo "Total data files: $(ls ../data/raw/*.csv | wc -l)" >> summary.txt
echo "Total participants: $(cat ../data/raw/*.csv | grep -v subject | wc -l)" >> summary.txt
cat summary.txt
```

Question to answer: Did this create a useful summary?

Expected outcome

You automated summary statistics. This approach scales - if you add 100 more files, the command still works.

6.7.9 5.8 Find Files by Content

Task:

```
cd ~/psych_study
grep -r "S001" .
```

Question to answer: What does `-r` do with `grep`?

Expected outcome

`-r` makes `grep` recursive - it searches in current directory and all subdirectories. Useful for finding where specific data appears.

6.7.10 5.9 Check Disk Usage

Task:

```
cd ~/psych_study
du -sh *
```

Question to answer: What does this tell you?

Expected outcome

`du` = “disk usage” `-s` = summary for each item `-h` = human-readable (KB, MB instead of bytes)

Shows how much space each directory uses - important for large datasets.

6.7.11 5.10 Final Cleanup Exercise

Task: Time to clean up our practice.

```
cd ~
ls terminal_practice # Verify it exists
rm -r terminal_practice # Remove it

ls psych_study # Verify it exists
rm -r psych_study # Remove it
```

Question to answer: Why is it important to be careful with `rm -r`?

Expected outcome

You successfully cleaned up practice directories. Always double-check paths before using `rm -r` - there’s no undo!

6.8 Self-Assessment Checklist

After completing these exercises, you should be able to:

- ☐ Navigate the file system (`cd`, `pwd`, `ls`)
- ☐ Create and remove files and directories (`touch`, `mkdir`, `rm`, `rm -r`)
- ☐ Copy and move files (`cp`, `mv`)
- ☐ View file contents (`cat`, `less`, `head`, `tail`)
- ☐ Search within files (`grep`)
- ☐ Count and sort data (`wc`, `sort`)
- ☐ Use pipes to combine commands (`|`)
- ☐ Use wildcards for batch operations (`*`, `?`)
- ☐ Navigate command history (arrow keys, `Ctrl+r`)
- ☐ Use tab completion
- ☐ Create simple shell scripts

- Make files executable (`chmod +x`)
-

6.9 Common Pitfalls and Tips

6.9.1 Safety

- Always check `pwd` before running `rm -r`
- There is no “Undo” in the terminal
- Test commands on unimportant files first

6.9.2 Efficiency

- Use Tab completion religiously - prevents typos
- Use up-arrow for recent commands instead of retyping
- Learn `Ctrl+r` for searching history - it’s incredibly useful

6.9.3 Debugging

- If a command doesn’t work, check for typos (especially spaces and `-` vs `_`)
- File names with spaces need quotes: `"my file.txt"` or escaping: `my\file.txt`
- Case matters: `File.txt` `file.txt`

6.9.4 Getting Help

- `man <command>` shows the manual (press `q` to quit)
 - `<command> --help` usually shows brief help
 - Google “bash [`command`] examples” for more information
-

6.10 Next Steps

Once comfortable with these basics, you might explore: - **Text processing:** `awk`, `sed` for data manipulation - **Version control:** `git` for tracking changes - **Remote access:** `ssh` for working on servers - **Scripting:** Writing longer bash/zsh scripts for automation - **Package managers:** `brew` (Mac) or `apt` (Linux) for installing software

Remember: The terminal seems arcane at first, but it’s actually more logical than most GUIs once you understand the patterns. These commands have remained largely unchanged for 40+ years because they work well.

6.11 Getting Stuck?

If you get stuck or your terminal stops responding: - **Ctrl+c** cancels the current command - **Ctrl+d** exits many programs
- If completely stuck, close the terminal window and open a new one - Type **reset** if your terminal looks weird/corrupted

License: This document is released under CC BY 4.0. Feel free to adapt for your courses.

Chapter 7

Version Control

7.1 Git

7.2 An old overview of git and Github

<https://vimeo.com/456349738>

7.2.1 Getting Git

It depends on what system you have. See the operating system appendices.

7.2.2 Configuring Git

Do this in your terminal (and please use your own name and email address).

```
git config --global user.name "John Doe"
```

```
git config --global user.email johndoe@example.com
```

7.3 Other version control system

1. Mercurial
2. Darcs
3. CVS
4. Subversion
5. Pijul

7.4 Git is not Github

And github is not the only way to share code and data using `git` version control.

1. OSF.io 2. Sourceforge 3. Bitbucket 4. Gitlab The university provides you with a gitlab account: <https://git.uwaterloo.ca> 5. Codeberg

7.5 Using git with github

You will need an ssh key. And the easiest way to get an SSH key is via the terminal. If you haven't done any of the terminal exercises, now might be a good time to go there, do at least the basics so you know where your terminal is, and then come back here to finish the set up.

The official instructions

7.5.1 Creating an ssh key with your terminal

Detailed instructions on ssh-keygen

A slower walk through with some github specifics

7.6 Understanding Git and the Workflow in Pictures

Git can be confusing.

Clone or fork?.

A long video where I try to talk and draw through the concepts. <https://vimeo.com/456349595>

7.7 Principal Terms Terms for Git

- **Fork:** A copy of one github repository to another **github** repository.
- **Clone:** A copy of one **git** repository to another **git** repository.
- **Remote:** This is a repository that you are following. You will typically *pull* from these, but your *push* permissions may be limited depending on the distinctions between forks and clones, and whether you own the remote or someone else does. You can have more than one remote.
- **Pull:** the transfer of information and changes **from** one repository incorporated into another. This is how you get the new information from a remote transported to a local repository that you control.
- **Push:** this is the transfer of information **to** a repository you control (or have permissions to push to) from another repository that you control. This is often from your local laptop version to the hosted repository (your fork) on github.

- **Pull Request:** When you have information or changes that you think would be helpful to a remote you do **not** have push permissions for then you can request that the owner of that repository pull in your changes. This is a formal process called a pull request. It is primarily a github concept and not a git concept.
- **Branch:** within a repository the development of the code may be proceeding in a few different directions at the same time. The principal production branch is conventionally called **master**. And the principal repository that is the main, shared one is called **origin**. We will not be working with branches in our course, but those terms do show up in commands.

Chapter 8

Lab Notebooks

I will not actually be spending much time on this, but we could. It is a neglected practice in psychology to keep a lab notebook that is updated. Here are some options if you are interested in exploring this topic.

- Jupyter notebooks in Psychology
- 10 rules for lab notebooks (electronic)
- How to keep a lab notebook
- Pick an electronic lab notebook
- Lab notebook 2023 guide

Chapter 9

Writing Scientific Documents

Writing scientific documents is more than writing papers. It can also include posters, websites, books, and all of these may require images, graphs, references, formulas, and even code. We will take a look here at some of the tools that can help with this, and later we will look in more detail at some of the options.

9.1 IDEs and Writing up your Research

Class Question

What does IDE stand for and what are examples of IDEs?

9.1.1 VSCode

There are many powerful IDEs available. It will be useful for you to try more than one. However, at this moment in time the clear winner of the IDE popularity contest is Microsoft's Visual Studio Code.

Since I want you to learn to use your computer as a tool, and to continue to be able to do so when tooling changes, you need to be able to:

1. Locate code/programs on the internet
2. Download and install that program to your computer
3. Locate the help to begin to use the tool without formal instruction.

 In-class Exercise

Locate, download, install, and verify you can start VS Code.

9.1.2 Some helpful links to get you started

- Basics Video
- Using VSCode with Python
- Using VSCode with R
- Text Editor Page from Quarto Site
 - What is “pip”? And why will you need to know?

Additional material about VS Code and its use as an IDE will be found in this section quit ### Overleaf

VS Code is far from your only option. For technical writing and collaboration many scientists rely on Overleaf.

Nature will [let you submit your manuscript](<https://www.nature.com/srep/author-instructions/submission-guidelines>) from an Overleaf template.

You always have choices. You can even [write LaTeX with Quarto](<https://github.com/James-Yu/LaTeX-Workshop?tab=readme-ov-file>) (see also).

 Classroom Discussion

What is single source publishing? How does this idea impact your preference for tools like Overleaf and Quarto?

9.1.3 RStudio

RStudio is a popular IDE for writing R code. R is a programming language particularly useful for statistical analyses. While initially RStudio was only for the R language the number of programming languages it currently supports has grown. It still is not as widely useful as VS Code, and many of RStudio’s features are designed for the interactive evaluation of data, and so it has less of the cutting edge features that software developers expect in their code tools. Whether you want to use RStudio or not will depend in part on your planned activity.

9.2 Scientific Publishing Tools

To understand the plethora of contemporary tooling available to facilitate writing you should know generally what is meant by a mark-up language. This will help you evaluate whether it makes sense for you to switch from a word processing program, like Word for Windows, to a more general tool like VS Code or emacs.

Some of the available options you should be familiar with are

- LaTeX
 - LaTeX This was the standard for several decades, and used more by mathematicians than others, but today it seems limited in that it produces mostly pdfs, and many people also want their files to export to html (webpages)
- A guide to LaTeX for Word Users (pdf)
- The descendants of markdown
- Quarto/Qmd
 - Quarto (we will skip Rmd and since we are using its successor qmd. Both are related to the people who wrote RStudio, `knitr`, and are now known as Posit).
- Org Mode
 - Org Mode works particularly well with Emacs and allows you to embed code from multiple languages.

9.3 Things You Can Do With Quarto

Here are some of the helper links for things we can do with Quarto:

- journal formats try the Elsevier format for this course. A lot of psych journals are Elsevier owned.
- presentation
- Talking to databases.
- Make a website/blog for your work or lab
- And more ...

9.3.1 Making a Website with Quarto and Github Pages

As science progressively moves to online communication there is a need to have a place to show case your work and share ideas. One way to do this is with a personal website structured around your scientific interests and activities. While there are many methods, tools, and hosts for this I will focus on sharing how you can use the Quarto publishing tool.

9.3.1.1 Hosting

To have a website viewable by the world it must be hosted on a server that is viewable by the world. For example, it cannot be behind an institutional firewall (as are most of the computers at the University of Waterloo). And if you host on a University or Company server you have the potential of being limited in what you can share, how often you can update, or you may be required to adhere to certain styles and formats mandated by the institution's brand. One way to avoid this is to find an external hosting site. A modest fee would allow you to have a virtual server in the cloud, but the example we will use here to get you started is a free service available to those with Github accounts: Github Pages.

9.3.1.1.1 Testing Github Pages

Assuming you have an account you should be able to create a minimal webpage in a few steps.

1. Go to personal dashboard and create a new repo as instructed on the Github Pages website. The repo must be named `.github.io`.
2. Use terminal to clone this repo to your local computer.
3. `cd` into the correct directory using your terminal (inside VS Code is fine). And using the *terminal* do `echo "hello world" > index.html`.
4. Do the usual add, commit, and push and verify your web site works at `<username.github.io>`.

You now have a website!

9.3.1.2 Blog Project in Quarto

The advantage of managing your blog (or website) in Quarto is that you get to take advantage of all the markdown language features. This is much more convenient than writing in html directly. You can also embed code and output just as you have been doing when you export your R code to html.

There are several ways that the Quarto people recommend getting the material to and from github. The one that I found most convenient (that is the one that I got working with the least amount of swearing) was to configure a project that exports to a `docs` directory.

Creating the blog project in Quarto.

1. In the **terminal** move to the directory with your repo.
2. Create a branch called `gh-pages`.
 1. You can find the steps for creating this branch at [blog](#), but for the later steps he uses the `publish` option where we will output to `docs`.
 2. Make sure everything is committed or stashed before you begin then:

```
git checkout --orphan gh-pages
git reset --hard
git commit --allow-empty -m "Initializing gh-pages branch"
```

Caution

If you are going to try the “publish” option you will not want to edit in the `gh-pages` branch. That branch will just be used for publishing your content. However, when using the `docs` approach, as I am illustrating, it is fine to just work on the `gh-pages` branch directly. For the `docs` method to work properly you must go into your settings for this repository on Github and find the Pages area. Then make you set the branch correctly and point it to the `docs/` directory.

3. Add the following to your `.gitignore`:

```
/.quarto/  
/_site/
```

If you don't have such a file, create it. If you don't know what a `.gitignore` file is ask!

4. Now we use the `quarto` command in the terminal to initiate our website as a project. This creates the basic structure we need. We can later change or adapt it.

```
quarto create project blog .
```

When I did this it asked me for a title and then it asked to open everything in vs code. Then you will want to edit the `_quarto.yml` file to direct the output to the `docs/` directory. The top of you file should look like

```
project:  
  output-dir: docs/  
  type: website  
  
website:  
  title: "brittAnderson.github.io"  
...
```

5. Try `quarto preview` to see the current skeleton.
6. Explore the files to see what you can change. Try the about page for a starter. Can you add a picture of yourself, and make it say your name?
7. Try adding a post. First study the structure of the existing posts. You will need to emulate this in order for things to track correctly.
8. Then publish `quarto render` This command will compile the html versions of your qmd code and direct them to the correct places. Then if you have set up thing correctly you can use the git features within VS Code to commit and push your changes. After a few minutes they should be available via your github link.
9. My files are at Britt's Github Example and the demo is at brittanderson.github.io.

Note added Sept 2025. A good article on academic websites is to be found here: <https://doi.org/10.1038/s41562-025-02310-6>

9.3.2 Journal Article Authoring



Classroom Exercise and HW

Write an article with quarto/qmd and using any of the linked article formats. Each has instructions on their use. Your article should look similar to your overleaf submission. All the components of an article, at least one figure, and at least one reference. Instead of using an image from the web try to include either your R or Python code for the figure you liked.

I find this easier to do from a terminal.

```
#!/ eval: false
```

```
quarto use template quarto-journals/elsevier
```

This will result in several questions for you to answer. When you are done you have starter versions of all the pieces you will need to get going.

9.3.3 Presentations

Presentations are a standard component of professional life. Despite that importance the materials are often developed in a casual way that does not give the best impact to the material. There are many more versatile tools than contemporary Power Point. Though power point can make an effective presentation its default use does not typically achieve that goal. Quarto provides easy access to two other presentation tools that will expand one's recognition of what an effective presentation tool can do.

9.3.3.1 Reveal.JS

Reveal.js is a javascript library for the creating presentations that work as well on the web as they do in person. Quarto also supports Reveal.js.

9.3.3.2 Beamer

Beamer is a package for LaTeX that allows professional quality typesetting. It is particularly relevant for presentations that have a lot of mathematical material, though it is quite capable of supporting any general presentation as well. It is less flexible for the web since it produces PDFs. These can be shared on the web, but they require the intermediate step of downloading them rather than just playing them. Beamer also has a “Poster” variant that can be used to make high quality academic posters.

Overleaf has a beginners tutorial. Quarto also supports Beamer.

LaTeX and Knitr in Overleaf[[knitr]] with overleaf. A scientific poster with Overleaf?

9.3.3.3 Academic Posters

Typst

[Scientific Posters with Quarto](<https://github.com/quarto-ext/typst-templates/tree/main/poster>)

Chapter 10

Coding General

Simply put, this is you telling the machine what to do. It is usually done via the writing of a file in plain text that is then processed in various ways to yield machine code that the computer actually uses. It is possible to program in machine code, but it is not very user friendly.

10.1 Languages

There is more than one language available for programming your computer and each has their pros and cons. Some are “low level” or close to the metal (see for example, rust or zig), while others, the ones we will use in this course are considered high level, and abstract away many of the details (such as how much memory to allocate or how to clean-up (garbage collection) unused data from memory).

When evaluating the fitness of a new programming language ask yourself not only whether it meets your needs today, but does it look likely to be available tomorrow, or five years from tomorrow. It is also, in general, to chose a more general purpose language that you can use for more than one particular task. The latter will allow you to leverage your knowledge of how to get things done in one domain for getting things done in others. And it is likely that you will learn general patterns that may help you pick up additional programming languages more quickly if, in the future, you need to change.

10.2 Common Programming Constructs

10.2.1 Variables

Variables are things (lists, values, names, and such) that you want to give a name to. You might do this because you think you are going to want to do

something with them later and it might be more convenient to have a short, memorable name for referring to them. **### Functions** Functions have inputs and output. The inputs may be called parameters or arguments. The outputs maybe called return values. What the function does is describe the processing of the inputs to the outputs. Think of a factory. It takes in raw materials (inputs) and produces an output. You are describing a factory when you write a function. **### Interpreted** An interpreted language is one where you type in code one line at a time and a program converts that to machine code, evaluates it, and prints the return value for you to read. This is an interactive mode. Python is an example of an interpreted language. **### Compiled** You write the whole program. Another program reads your code and all at once, and possibly requiring several passes and linking to other code elsewhere produces a single compiled version for the machine to read and use. C is an example of a compiled language. **### Packages/Libraries**

Many programming languages are very basic in what they offer. In order to do the cool stuff you will need additional collections of code, some of it may even be written in a different programming language, often called packages or libraries that you import, source, or load. `ggplot2` is an example of an R plotting library. `matplotlib` is similar for python. **### Types**

Many programming languages have a notion of a datatype. 1 can be a number, but “1” can also be a character that you are typing. It doesn’t make sense to add one, the number, to one, the character, but it might make sense to `1 + 1` for either. You would expect 2 for numbers and maybe “11” for characters. Some programming languages, such as python, type *dynamically*. They figure out what type to treat a value as as you type. They try to deduce it from the context. Other languages, such as Haskell, are *statically* typed. They require that everything have a type and that it never changes. Dynamically typed languages are often easier to get started with, but statically typed languages can prove helpful when writing larger pieces of software.

Other types of data structures that you might run across in this course are: - Lists, - Tuples, - Data.Frames

10.2.2 Loops

Loops are a common programming construct. They allow you to repeat an action multiple times. For example if wanted to print a number you could repeatedly use something like `print(my_number)` as long as you changed `my_number` in between each use.

Common varieties of loops are the `for` and `while` loop.

10.2.3 Virtual Environments

You will often have software installed on your computer that you use for multiple purposes. There is also software on your computer that is used by your operating

system. If you download different versions of that software for your own use you may find that your programs do not run correctly, or even worse that your system does not work properly. Do avoid such problems you may want to “sandbox” your development environment. Conflicts are less common for R, but very common for python. Such sandboxes are often called `venv`. A virtual environment lets you install software that only works locally, and cannot interfere with your global system software or other virtual environments you may have constructed for other projects.

10.2.4 Assignment and Equality Are Different in Programming

Assignment means you are assigning a name to something. Equality means that you are testing if two things are the same. One equals sign does not equal two equals sign. That is `=` `!=` `==`. Exclamation points are often used to *negate* something (turn true to false and vice versa).

```
a = 2
print(a == 3)
```

False

In the first line we *assign* the value of two to the variable `a`. In the next line we test if `a` and `3` are in fact equal to each other. A “boolean” value is returned: `false`.

Chapter 11

R

11.1 Installation

See the operating system specific appendices.

11.1.1 Test your installation

```
R cmd --version
```

```
R version 4.5.2 (2025-10-31) -- "[Not] Part in a Rumble"  
Copyright (C) 2025 The R Foundation for Statistical Computing  
Platform: x86_64-pc-linux-gnu
```

```
R is free software and comes with ABSOLUTELY NO WARRANTY.  
You are welcome to redistribute it under the terms of the  
GNU General Public License versions 2 or 3.  
For more information about these matters see  
https://www.gnu.org/licenses/.
```

You should see some information about your R version.

11.2 Interfacing with VS Code

Restart VS Code and install the R extension (look to the left panel for the extension launcher; it looks like a little set of squares).

11.2.1 Resources

R for Data Science is a text book on data science that uses R. The author is a prolific R package author and the itself written in Quarto. It includes many examples that you can test by cutting and pasting into your IDE.

11.3 Installing R Packages

There are many tools for this. Some of them are GUI tools, and they will at first appear easier. But if your library starts to grow, or you need to install particular versions of packages, old packages, or packages from Github and not on CRAN, then you will be much happier in a R interpreter.

To create an R interpreter in VS Code you can simply launch a new terminal and then type `R` and `.`. You should see an R welcome message. Code like the snippet to follow shows a basic installation exchange.

```
install.packages("tidyverse",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
library{tidyverse}
```

Figure 11.1

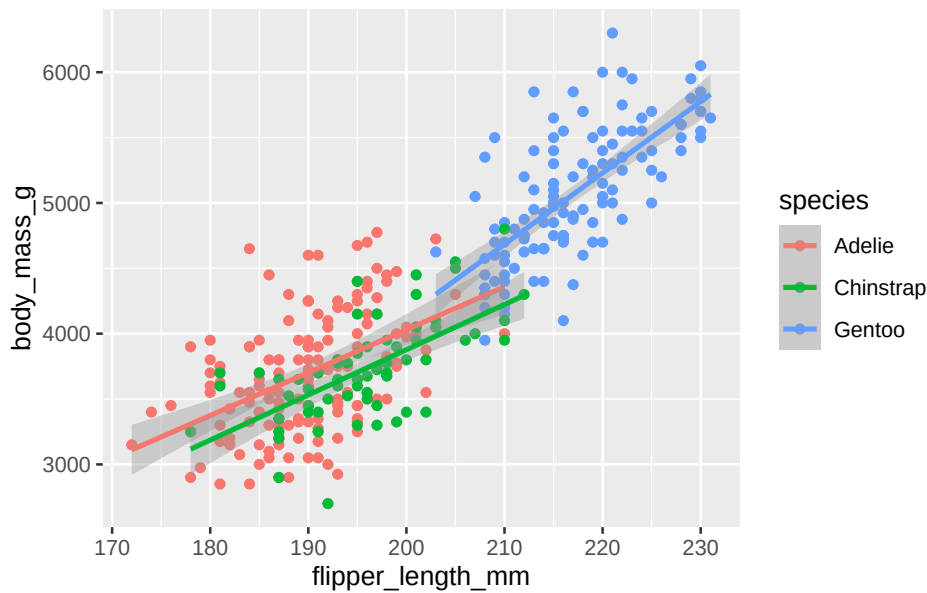
The tidyverse is a very popular collection of R code that itself will depend on many additional pieces of R code and other packages, some R and some not.

11.4 Mix R Code with Text for Reproducible and Documented Workflows

Here is an example of embedding R code in a qmd document. In this code we download and install a package if necessary (the if statement). Then we use that package to load some data and generate a plot. The plot you see in this document is the result of that code. I did not cut and paste a png.

```
if(!require(tidyverse)){
  install.packages("tidyverse",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
}
if(!require(palmerpenguins)){
  install.packages("palmerpenguins",repos='https://mirror.csclub.uwaterloo.ca/CRAN/')
}
library(tidyverse)
library(palmerpenguins)

ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)
) +
  geom_point() +
  geom_smooth(method = "lm")
```

11.5 R Types

As mentioned in the code chapter, programming language values are typically typed. R in particular has many complicated data types that represent the output of functions and statistical analyses. If things are getting confusing you might want to check on the type of your data.

```
a = 1
typeof(a)
[1] "double"
```

Figure 11.2: Checking the type of a number in R.

```
tpl = c(1,2)
lst = list("firstName" = 'Britt', "lastName" = 'Anderson')
df = data.frame('fn' = c("bob","jane","griffin"), "gndr" = c('m','f','o'))
df
```

	fn	gndr
1	bob	m
2	jane	f
3	griffin	o

Figure 11.3: Each of three datatypes in R are demonstrated.

11.6 Data.Frames

Data.Frames are particularly central to the R analysis model and are a killer feature. Think of **data.frames** as a supercharged spreadsheet with column names.

```
is.data.frame(cars)
```

```
[1] TRUE
```

```
head(cars)
```

	speed	dist
1	4	2
2	4	10
3	7	4
4	7	22
5	8	16
6	9	10

cars is a data set built in the R language. We use the **is.data.frame** command to ask if a variable is or is not a data.frame.

11.7 Selecting Data

A bit of code that tests for whether something is true or false can be called a *predicate*. Predicates are important when having your code do something conditionally. If this is true then do this, otherwise do that. This common construct of “if-then-else” exists in most programming languages. In the next code snippet we use the conditional expressions to select on a subset of our data. You might use something similar to figure out how many of the participants in an experiment were Arts students and under 20 years of age.

```
length(cars$dist[cars$speed > 10 & cars$speed < 20])
```

```
[1] 29
```

Figure 11.4: How many cars are there that can go faster than 10, but not more than 20?

What is going on here? 1. **cars** is the name of the data frame. 2. We access a *column* of the data frame with the dollar sign notation **cars\$dist**. You can see the names of the columns in a data frame with **names(cars)**. 3. We use the square brackets to *index* the column of data we are interested in. Here we do not use specific numbers, but rather we use a *boolean* to compare the values in a column and we only include the *TRUE* values. Here we ask for all the indices where the speed is greater than 10, but less than 20, and we use those indices to get the values for the **dist** column.

11.10 More Practice

1. Edit the above so that it prints the individual number each time it goes through the loop, and not the whole list.
2. Create a list of at least 8 individual characters. 1. Make sure they are **not** in alphabetical order 2. Print the letters one at a time. 3. Print the letters sorted alphabetically one at a time, but *do not* overwrite your original list. 4. Print the letters from both lists with a **format** command that says which position the letter is in.

```
myName = "brittAnderson"
myList = unlist(strsplit(myName, ""))

for (l in myList){
  print(l)
}

for (l in myList[order(myList)]){
  print(l)
}

i = 1
for (n in order(myList)){
  t <- sprintf("The %.0fth letter of myList is: %s, but is %s in the sorted list.", i, l, myList[order(myList)[n]])
  print(t)
  i = i+1
}
```

11.11 Writing a Function in R

```
myadd <- function(x,y) {
  return(x+y)
}
```

- ① The arrow is used to assign something to a name. Other languages usually use the equals sign. The keyword **function** tells R you are creating a function, and the parentheses following establishes the number of inputs and the nicknames you will use in your function.
- ② **return** specifies what we want to get back from the function.

Now we can use the function we defined with particular inputs.

```
myadd(2,3)

[1] 5
```

Chapter 12

Python

12.1 Installation

See OS specific appendices.

12.2 Packages

Unlike R, you do not install python packages from within an interpreted environment. Worse, the python package system has different tools. Recently, `pip` has been the most commonly used, and there is an interface to it through `uv`. Conflicting packages are a real risk. Use a virtual environment. Also, available via the `uv` interface.

12.2.1 Using `uv` to create an env and install local packages

First, initiate the virtual environment and declare the python version to use. You will first create the environment. Then you will **activate** it. I find this far easier in a shell/terminal/command-line environment.

```
#| eval: false
```

```
uv venv --python 3.10
source .venv/bin/activate
```

NB: As of 2025 the most recent version of python that works well with psychopy is said to be 3.10.

This will create a new *hidden* (directories starting with a “dot” like `.venv` will not display on many operating systems unless you specifically ask to see them.) directory called `.venv` inside the directory where you invoked that command. To activate the virtual environment you do `source <path-to>/.venv/bin/activate`.

When active your subsequent python command should refer to this python installation.

To install packages the code would look like (again in the shell):

```
#| eval: false  
  
uv pip install numpy matplotlib jupyter pandas
```

I picked these packages, because I know they will come in handy later.

12.2.2 Testing Your Python Package Installation

Warning

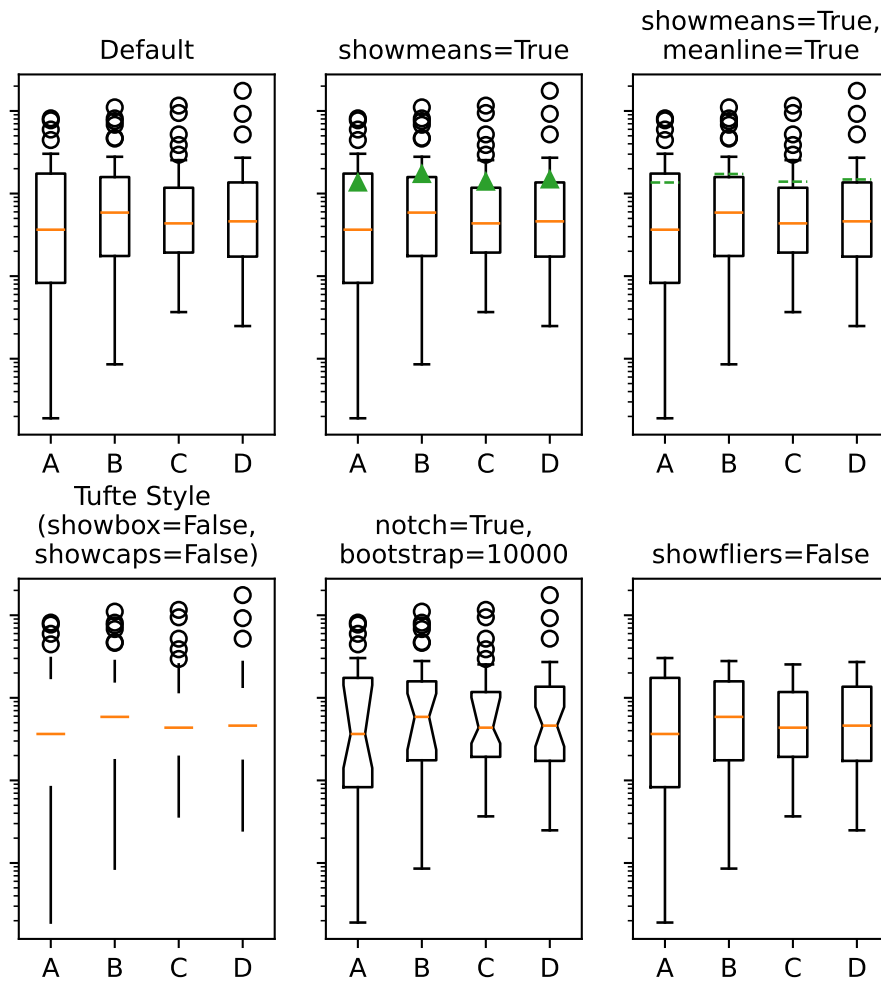
You may need to install the “reticulate” package for R if you want to run both python and R code in the same document as I am trying to do here.

And you will.

Warning

Use your .venv. If you are using VSCode you will have to tell VSCode to use your python environment. The shortcut Ctrl-Shift-p will open up the command menu and you can search for python: select interpreter. Guide it to the python version in your *activated* venv.

If you did all the above then this should generate a rather cool looking set of figures.



Warning

Python cares about whitespace. It is strongly recommended that you only use spaces and not tabs when writing python code.

12.3 Loops in Python

This is a snippet of code to demo the use of a for loop in python. Note how we can use the `enumerate` function to return two loop variables (`i` and `j`) that we set at one time.

```

my_name = "britt"
my_name_as_list = list(my_name)
for i, j in enumerate(my_name_as_list):
    print((i, j))

(0, 'b')
(1, 'r')
(2, 'i')
(3, 't')
(4, 't')

```

Some things to note: 1. Assignment in python does not use the arrow like R does. 2. Strings and lists are different *types* of data. I have to coerce my string to a list.

Figure 12.1: What a for loop might look like in python.

12.3.1 Loops in Python Make Use of Iterables

Python refers to things called “iterables.” To iterate is another way of saying that you can keep doing the same thing over and over. Imagine a bowl of ice cream. It is “eatable”. You take one spoonful, and then another, and keep taking spoonfuls until the bowl is empty.

Indexing is how you get the location of an element in a list. Think of the row or column number in a spread sheet program, but indexes can be more powerful, and can be nested easily. In many programming languages, but sadly not all, **indexes start at 0**. Different programming languages will have slightly different syntax.

```

name_dict = {'firstName' : 'Britt', 'lastName' : 'Anderson'}
my_list = list(range(1, 10))
print(name_dict['firstName'])
print(my_list)
print(my_list[0])
print(my_list[-1])
print(my_list[0:4])

```

12.3.2 Additional Examples of Programming Constructs in Python

Another for loop in python

```

for ml in my_list:
    print(ml)

for i, ml in enumerate(my_list):
    print("The {0}th element was {1}".format(i, ml))

```


- ① Notice that here we use `range` where we use `seq` in R. There is usually a way to do something similar in both languages, but they may not have the same name.

```
Britt
[1, 2, 3, 4, 5, 6, 7, 8, 9]
1
9
[1, 2, 3, 4]
```

Figure 12.2: The use of the `-1` as an index is a python trick for getting the last element in a list. Think of the list as a circle. If you count backwards from a list you will get to the beginning eventually (index 0) if you went back one more step (`-1`) you would circle back to the end of the list. Test what happens when you try `-2`. Does it keep going? A lot of learning how to program is just doing stuff to see what happens.

```
1
2
3
4
5
6
7
8
9
The 0th element was 1
The 1th element was 2
The 2th element was 3
The 3th element was 4
The 4th element was 5
The 5th element was 6
The 6th element was 7
The 7th element was 8
The 8th element was 9
```

Conditionals in python

```
if 2 == 3:
    print("Wha....?\n\n")
elif 3 == 2:
    print("Now that is odd")
else:
    print("2 does not equal 3.")
```

2 does not equal 3.

Note that we use colons and white space to organize code in python, not the `{}`

of R.

! Important

Python uses a different syntax to define functions from R.

```
def my_add(x, y):  
    return(x + y)
```

12.4 Popular and Useful Python Libraries

1. Numpy
2. Scipy
3. Matplotlib
4. Pillow
5. Sympy

Chapter 13

Programming Experiments

One of the most common uses of computers in psychology is for the programming of experiments. Classic experiments often involve a combination of visual stimuli that the participant has to respond to in some way. Typical responses are often button presses or mouse clicks, though many other varieties of stimuli and reports are used. While many programming languages can be used for this task we will begin with one of the most popular, Python. And if time permits we can explore some examples with Javascript as the world of experimental psychology is increasingly complementing the usual lab based testing paradigm with online studies.

13.1 All-in-one options

Excellent tools exist that provide turn-key solutions. A very popular one is Psychopy. This package is available for the popular operating systems and gives a gui like builder that allows you to program visually. It offers the opportunity to output your experiment into a web friendly format that can integrate with an online hosting service for performing experiments. However, my experience is that while such tools make it easy to get started they don't help people develop the general skills that will transfer into programming that distinctive experiment that no one else has done. And the knowledge you gain is often interface specific. It is not general, and so your learning cannot transfer into writing R code or porting your experiment to MATLAB if that becomes the tool a collaborator wants to use. Even if this is the sort of tool you eventually settle on for the very reasonable reasons that your experimental needs are not complex and the quality of the implementation allows you to get work done more easily and on board new investigators more quickly, it will still make you better at planning and handling problems if you have a knowledge of what is going on under the hood.

13.2 Python

The following examples focus on the sort of experiment where you want something to appear on the screen, and you want feedback from the participant. Current computer graphics don't work like they did twenty years ago when we all had heavy CRTs on our desks, but one can still deploy some of the intuitions of that era.

The monitor is a screen and it is displaying a picture. In the background what is to be shown next is in preparation and at some point what is on the display now will be replaced by the next collection in the queue. Even though we no longer literally raster the display from top to bottom and side to side you can still usefully think in terms of *frames*. Thus, almost all your experiments will need the mechanics of a window the participant views and in which you can write things you want them to see.

13.2.1 Pygame

For practice I will use the pygame python library. There are a large number of help and tutorials for this well established library that has been in use for decades. It is well vetted and documented, and relies on a simple direct library for video manipulations.

13.2.1.1 Some Links for Getting Started with Pygame

1. The pygame getting started wiki. Most of my examples will draw from this site.
2. Real python pygame tutorial
3. Game Development Academy

13.2.1.2 Installation

A virtual environment is recommended, but not necessary. If you are using a venv activate it first.

Then

```
pip install pygame
```

①

```
python -m pygame.examples.aliens
```

②

- ① This line installs the library.
- ② This launches a test game. If you see the game, you have installed pygame. I find this game plays a little fast on modern hardware.

One of the better ways to get started is to gradually hand in functionality one step at a time until you have the basics of what you need on a trial and then to refine that to loop through the multiple trials you will probably need.

```

import sys,pygame
pygame.init()

mywindow = pygame.display.set_mode([800,800])
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    mywindow.fill((128,15,15))
    pygame.display.flip()

pygame.quit()
sys.exit()

```

Figure 13.1: This python program will, assuming you have installed pygame, launch a window and close it when you click the “x”. Can you change the window size and background color?

Here is an example of a bit of code that shows some elementary use of the mouse, response time, shape and text functionality.

```

from itertools import compress
import sys,pygame
pygame.init()

my_font = pygame.font.Font(None,72)
test_text = my_font.render("Hello P390", 1, (200,10,200))
test_text_pos = test_text.get_rect(centerx = 400)

#Helper functions
def which_buttons (bs):
    return(list(compress(range(len(bs)),bs)))

#button colors
clrs = [(0,0,255),(0,225,0),(225,0,0)]
actv_clr = 0
rslts = {'trial_num': -1,'color' : None , 'time' : None}

mywindow = pygame.display.set_mode([800,800])
running = True
myrt_clock = pygame.time.Clock()
t1 = myrt_clock.tick(60)
anew = pygame.mouse.get_pressed()
trl = -1,

```

```

clr = clr[0]
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    mywindow.fill((128,15,15))
    pygame.draw.circle(mywindow, clr[actv_clr], (400,400), 75)
    mywindow.blit(test_text, test_text_pos)
    pygame.display.flip()
    aold = pygame.mouse.get_pressed()
    if(any(aold) and (aold != anew)):
        t2 = myrt_clock.tick(60)
        rslts['time'] = t2 - t1
        rslts['trial_num'] = rslts['trial_num'] + 1
        actv_clr = which_buttons(aold)[0]
        rslts['color'] = actv_clr
        anew = aold
        print(rslts)

pygame.quit()
sys.exit()

```

13.2.1.3 Next Steps

You will want to save your data. As a first step see if you can open a file, and the if you can use the dictionary created to add lines to a csv file.

13.3 Javascript

WIP

Chapter 14

In Lab Experiments

- Making stuff appear on monitors.
- What is OpenGL?
- Use pygame to make some simple visual experiments.
- Have a beauty contest the following week to see what people have been able to make?
- Make sure that people are using venv

14.0.1 Online Experiments

Running a server for testing and more? XAMPP Running a lab now seems to require some familiarity with servers. And people who want to write their experiments in javascript often want to try things out first so it seems something like XAMPP might be a good resource. *Exercise* to download and get the XAMPP server running? Need to expand this. An exercise with JavaScript? This site has a simple bit of code for throwing an image on the screen. Then use the XAMPP server to test it? Require changing the image? Animate a button to toggle or get a random image?

Chapter 15

Handling Data

Before we can analyze data, we have to have some data. Broadly speaking our data may come from experiments we run ourselves, or it may come from data that is shared with us. In the first case we have some freedom to choose the format. In the later, we simply have to take what we are given. Therefore, we need to consider both how to save and convert data that we collect from an experiment. And we also need to be broadly familiar with common formats of data storage that others may use.

15.0.0.1 Data Formats

- Plain Text
 - CSV
 - TSV
 - JSON
 - Pickle
-
- Dat (R)

Some advantages and disadvantages of each.

Plain text is easy to do and human readable. It usually will require a unique parser, and forces all items to be text. Thus, things like numbers lose their specificity, and things like lists can become painful to retrieve.

CSV and TSV view all data as rows and a unique character (commas or tabs) separates the cells of the rows. Another character, typically a new line (often seen written as `\n`) is what distinguishes one row from the next. These file types are, largely, human readable if the number of columns and rows are not too large. They are not as space efficient as other formats, but that may be a reasonable tradeoff when the size of the data files is small.

JSON stands for Java Script Object Notation. Although, as the name implies, it had its origin in javascript it is largely language agnostic and is a common format for the storage of data of all types. Its being used as a standard means it has wide support from many programming languages and that means there are good tools for exporting and storing for most programming languages. It can be quite flexible in the type of data stored. It is not friendly for writing by hand, but the use of tools makes it fairly easy to take experimentally collected data created in, say, a python dictionary and convert it to a json format.

```
import random as r
import json as j
a = {'name': 'John Doe', 'age': 24, 'rt': r.random()}
js = j.dumps(a)
js
'{"name": "John Doe", "age": 24, "rt": 0.9327247829857759}'
```

Figure 15.1: The `dumps` command will output the json version as a string.

Note there is a very similar python function without the terminal `s`, `json.dump`, that will save the json data to a file. Some other code is needed to create the file pointer. But once that is done, and the json data saved, you can read it back with `json.load()`

```
import json
a = {'name': 'John Doe', 'age': 24}
js = json.dumps(a)

# Open new json file if not exist it will create
fp = open('test.json', 'a')

# write to json file
fp.write(js)

# close the connection
fp.close()
```

Figure 15.2: A simple example taken from [StackOverflow] shows a working example.

A JSON tutorial can be found [here](#).

Pickling is a data serialization library specific to the python ecosystem. That makes it useful if you are coding your experiment in python, but not in any other language. However, it can be very efficient on space, and there is good support for it in the python language so many experimental coding libraries built around python use pickle for data storage.

This code from a [GeeksforGeeks](#) article provides a nice minimal working example.

```

import pickle

def storeData():
    # initializing data to be stored in db
    Omkar = {'key' : 'Omkar', 'name' : 'Omkar Pathak',
            'age' : 21, 'pay' : 40000}
    Jagdish = {'key' : 'Jagdish', 'name' : 'Jagdish Pathak',
            'age' : 50, 'pay' : 50000}

    # database
    db = {}
    db['Omkar'] = Omkar
    db['Jagdish'] = Jagdish

    # Its important to use binary mode
    dbfile = open('examplePickle', 'ab')

    # source, destination
    pickle.dump(db, dbfile)
    dbfile.close()

def loadData():
    # for reading also binary mode is important
    dbfile = open('examplePickle', 'rb')
    db = pickle.load(dbfile)
    for keys in db:
        print(keys, '=>', db[keys])
    dbfile.close()

if __name__ == '__main__':
    storeData()
    loadData()

```

When working in the R environment the `save` and `load` function provide convenient ways to save your data and variables in their current state so that you can make available in your next interactive session what was available in your current one. However, this may not be the most interoperable way to work with your data if you need to share with collaborators who may not use R or if you yourself are open to the possibility of changing your analysis software. R offers two main functions for reading and writing data called `read` and `write`. Each of these has several specialization that target particular file types (e.g. `read.csv`) and also allow a number of customizations (e.g. whether to ignore a number of lines at the top of a file if there is a plain text header). Usually one of these variants will allow you to read your data into R or write data to a file that can be read by others.

15.0.0.2 Considering Pandas and Interoperability with R

Pandas is a major python library and effort to provide cutting edge tools for statistics and analysis to match what is on offer elsewhere. In my experience they are not the first place that statisticians develop their cutting edge tools, R remains most popular, but for all else the choice between the two, R and Python, is really a matter of preference and comfort. Pandas is a very powerful tool and if you are more likely to use python in the future you will want to explore Pandas fully.

Pandas can usually be installed with `pip install pandas`. Because of the size and complexity of python installations a virtual environment is recommended.

Even if you are not thinking of doing your data analysis in python if you are writing your experiments in python you might want to take advantage of some of the pandas tooling to make saving your data convenient. But before we show a pandas method let's see a lower tech, more old school method.

It is possible to manually write a csv file in python using a print command where you explicitly coerce everything to a string and intercalate commas along the way. For example,

```
a = 1
b = "britt"
c = 2.3
mycolumnheadings = "a," + "b," + "c" + "\n"
mystring = str(a) + "," + b + "," + str(c) + "\n"
print(mycolumnheadings + mystring)

a,b,c
1,britt,2.3
```

shows how to create a string from one's data. It is complicated and error prone. It may work for a start or for a very small project, but it does not scale well. In this example I am just printing for display, but if this was being used to save experimental data we would want to use a file handler.

15.0.0.2.1 Opening and Closing a File in Python

```
a = 1
b = "britt"
c = 2.3
mycolumnheadings = "a," + "b," + "c" + "\n"
mystring = str(a) + "," + b + "," + str(c) + "\n"
with open('/home/britt/gitRepos/uwloolP390/p390/myfile.csv','w+') as f:
    f.write(mycolumnheadings)
    f.write(mystring)
```

Now you can see if you can read that csv into R.

```
mydata = as.data.frame(read.csv("/home/britt/gitRepos/uwloop390/p390/myfile.csv"))
mydata
```

```
   a      b      c
1 1 britt 2.3
```

15.0.0.2.2 A Better Way: Dictionaries

Python dictionaries (which have their counterparts in most programming languages though they may be called something different) are a convenient way to store data where you can give it a meaningful name. For example, in an experiment with numerous trials and conditions you could have a *key* in your dictionary for “trial no.” and another for “condition”. The *values* for these would then be updated as your experiment unfolded.

A first pass to using dictionaries is to blend this with a python library for CSV files. Then you can create the column headings from your keys and then each trial that you loop through you can use the dictionary to write a new row to your csv file. Some of the functions you might want to use are:

- `csv.DictWriter`
- `writeheader`
- `writerows`

If you set up and define your dictionary at the start of the file you only need to update the changing values as they occur.

Another approach that will start you getting some familiarity with python pandas library is to make a distinct dictionary for each of the trials in your experiment. Then you can concatenate these dictionaries to a list and use that list to create a pandas dataframe. That could then be used in analysis python code or for importing into R. Here is a short example to illustrate the principle.

```
import random as r
import pandas as pd

mylistofkeys=["name","v1","v2","v3","v4"]

mydictlist = []
for i in range(5):
    mydictlist.append({key:value for key,value in zip(["name","v1","v2","v3","v4"],["britt"] + [r.random() for _ in range(4)])})

df = pd.DataFrame(mydictlist)

df.loc[:, 'v3'].mean()

np.float64(0.6106250208662504)

myfilename = "mypandastest.csv"
```

```
df.to_csv(myfilename)
```

You can save pandas dataframes as a pickle file, but then you will not be able to easily read it into R. CSV remains the easiest and most direct way to do this. It is probably all you need for most use cases. Though there are other options.

```
rdf = read.csv("mypandatest.csv")
mean(rdf$v3)

[1] 0.610625
```

15.0.0.3 Some Analysis Tools

- Pandas TBD
- Some more R TBD

Chapter 16

Knowledge Management

This section deals with the issue of how you keep track of the information you discover and how you mine it for new ideas, associations, and connections. In this section we will consider the notion of the *zettlekasten* and some modern tools for emulating this technique.

For many contemporary adherents to the *zettlekasten* system their pole star is Niklas Luhman a social systems theorist who was an prolific publisher and who credited much of his fecundity to this technique. Essentially, the method is one of cross-referenced index cards, and historical precedents are easy to find.

Although there are different flavors to this “slip box” approach the consistent theme is that one reflect on their reading, summarize it, and connect it to other works and concepts in their data base. We can perhaps do this more easily ¹.

16.1 (Zettlekasten)

1. Org-roam

Org-roam is a tool for use within the Emacs ecosystem. Emacs is, to paraphrase the old joke, an operating system managing as a text editor. Emacs comes with a suite of functions (**org-mode**) built in that facilitate outlining and note taking (and many more things besides). Org-roam is an additional piece of software designed to interact with the Emacs/Orgmode combination to implement many *zettlekasten* features. It is an emulation, in Emacs, of RoamResearch a self-described tool for networked thought.

The online manual for org-roam provides a brief introduction and overview of the tool. Very, perhaps too, briefly you use various key commands to create nodes

¹It is a question whether the enhanced digital ease impacts effectiveness since key to the practice is the reading and re-reading of “cards” in the index.

in your network and link them to others. You include notes on entries in your bibliographic system, or free standing notes organized around particular concepts. These links can be viewed both forward (things they link to) or backward (things that link to them), and there are also graphical views. A brief demonstration of some of these tools will be provided in class.

2. Dendron - works with VS Code

This note-taking knowledge management tool is included here because it can interact well with the VS Code and Markdown approach we have been learning about through our use of Quarto.

One of the common features you will find for all these tools is the paring back of dependencies. Most rely on some sort of plain text system at their base. This means you can always just read your entries even if the fancy link and processing features break or prove unwieldy. This generally future proofs your “cards” and also often means that things can work faster and more robustly because there are fewer dependencies.

3. Obsidian

Obsidian is another very popular offering in the same space as the above. This one however will cost you money. I include it here because of its wide popularity. The overview page of the website provides a good general visual orientation to the offerings and features.

Classroom Exercise

Taking Dendron for a test drive.

1. Install Dendron from the VS Code Marketplace.
2. Fill out the survey and any extensions you want (I took all the recommended) and proceed to the tutorial.
3. This should like quite familiar to your. The `qmd` files we have been writing are really just augmented versions of this `md` format.
4. Use the terminal to figure out what directory your notes are being stored in.
5. Use Ctrl-L to create a new note then navigate back to the tutorial.
6. The authors keep talking about `vim`. Look up what `vim` is. Do you have `vim` or its more difficult cousin `vi` on your computer? Try to open one or the other in a terminal.
7. How do you creat a note hierarchy?
8. View the tree mode.
9. Use the Command Palette to view all the Dendron related commands, and at least read the next steps.

16.2 Reference Management

In this section we will look at tools for two important functions related to references for the papers we write and the research we do. How do we find the details for references that we may need to cite, and how do we ease the task of writing so that our references and citations are properly formatted and easy to update when we need to change the citation format for a manuscript.

16.2.1 Finding Citations

- Semantic Scholar

This tool is supposed to help you find related papers. I have not found it all that helpful in my daily use. Do any of you have success stories to share?

- OpenAlex Try finding my articles here, and see if you can find out what my opinion on *attention* is.
- Google Scholar

See if you can find any research articles on Albert Einstein's brain by Dr. Sandra Witelson of McMaster University. What I like about this tool is that you can also search for articles that cite particular articles. How many of the articles that cite Dr. Witelson also contain the word myelin?

- Consensus

This is a new AI Driven search engine. It has some very nice features, but do be careful. Ask it what it thinks about me and category theory. Then look at the references. It is mixing me up with other researchers.

- Undermind

I think this is a very nice new addition to the search environment, but your free use is very limited. In the paid version you can easily identify articles that seem central to a field of research.

Sign up and try your free search.

- Research Rabbit To be added.

16.2.2 More on Citation Management

As the web as grown as the principal compendium of scientific articles the community has had to deal with the facts that URLs (website addresses) change. How do we avoid dead links when searching for literature? At the moment, our best tool is to refer to the doi.

What does doi stand for?

Find the article with the doi: 10.1371/journal.pone.0152353

There are many tools for this, but a convenient look up is provided by Crossref.

If you keep your own reference database, you will often find that locating the doi is the easiest way to get that reference saved to your database.

The same problem that occurs for uniquely specifying research articles can also occur for uniquely specifying researchers. There is more than Britt Anderson out there. How would a reader know if I am the one who wrote a particular article?

What is an orcid?

After you know what one is. Get yourself one if you do **not** already have one.

16.2.3 The last thing you want to do ...

when managing references is to be cut and pasting them into your own home grown document. You want to use a piece of software that can easily keep the data updated and correct, and facilitate you easily being able to correctly cite and correctly format your references when you write. Again, there are many tools, but zotero has withstood the test of time and is a free offering well supported by academic libraries world wide.

Zotero is a piece of software, and for this section you ought to download it and find a few articles on the web, in google scholar, or arxiv that you can install in your database to have a few test articles to work with. The support page will give you links to a quick start guide. There are many ways you can use zotero with tools you have used before (e.g. linking it to a word document), but in a moment we will explore a couple of ways to use it with the Quarto document writing system.

Since we don't want to have to manually format all our citations and references, and then reformat them when we are forced to by needing to resubmit to a new journal we want to rely on the computer to fix such things for us. One contemporary tool that can make that easier is csl. A *csl* file is a file that specifies all the necessary details about how references should be formatted for a particular citation style. Since you will most often be working with apa see if you can find and download the proper csl file <https://www.zotero.org/styles>, and then at least one other csl type so you can practice how easy it is to switch from one style to another.

16.3 Using References With Quarto and VS Code

Since you will, at least for this course, be writing Qmd files you may want to use Zotero for reference management and connect [it to Quarto](<https://quarto.org/docs/visual-editor/technical.html#citations-from-zotero>). You will find details under the citation section of this webpage. In

addition you will see how doi's, crossref, and other tools can be used to cite while you write in quarto. This workshop also provides details on citation management in Quarto as well as RStudio. See if you can write a minimal stub of an article and try some of the citation management features in VS Code and Quarto.

16.4 Using References With Overleaf

VS Code is not your only option. Overleaf also use ".bib" files to hold references, though some of the other syntax is different. We have seen this before. Zotero will offer you the option to export references to a bib file which you can then use directly or if you need to submit a .bib file for a manuscript.

In addition you can link Overleaf and Zotero to make your writing easier.

Chapter 17

Testing Your Javascript

17.1 Run your own server locally

Running a lab now seems to require some familiarity with servers. And people who want to write their experiments in javascript often want to try things out first so it seems something like XAMMP is a good tool to know about.

17.1.1 XAMPP

XAMMP

17.1.1.1 Downloading

Download link

17.1.2 In Lab Experiments

- Making stuff appear on monitors.
- What is OpenGL?
- Use pygame to make some simple visual experiments.
- Have a beauty contest the following week to see what people have been able to make?
- Make sure that people are using venv

17.1.3 Online Experiments

An exercise with JavaScript? This site has a simple bit of code for throwing an image on the screen.

Can you use the XAMPP server to test it? Require changing the image? Animate a button to toggle or get a random image? WIP HERE

Chapter 18

Large Language Models

18.1 Run locally

Why? Own your own data.

Security and Privacy.

Research applications you may not want participants' data fed to openAI or other proprietary vendors.

18.2 How to?

There are many methods to running LLMs locally on your own hardware. I have chosen GPT4All. It can run under OSX, Windows, and Ubuntu (linux).

18.2.1 Key Features of GPT4ALL

According to the website <https://getstream.io/blog/best-local-llm-tools/>:

GPT4All can run LLMs on major consumer hardware such as Mac M-Series chips, AMD and NVIDIA GPUs. The following are its key features.

- Privacy First: Keep private and sensitive chat information and prompts only on your machine.
- No Internet Required: It works completely offline.
- Models Exploration: This feature allows developers to browse and download different kinds of LLMs to experiment with. You can select about 1000 open-source language models from popular options like LLama, Mistral, and more.

- Local Documents: You can let your local LLM access your sensitive data with local documents like .pdf and .txt without data leaving your device and without a network.
- Customization options: It provides several chatbot adjustment options like temperature, batch size, context length, etc.
- Enterprise Edition: GPT4ALL provides an enterprise package with security, support, and per-device licenses to bring local AI to businesses.

It is also a very popular LLM for self-hosting (second only to Ollama, which you should also try out).

18.3 How-to

From the GPT4all download page download the version for your operating system.

Follow the download instructions appropriate to your system and pay particular attention to any warnings and make sure that you have sufficient freespace. This is a big application and the model parameters are also quite large.

After you have successfully downloaded and installed the program checkout the quickstart guide.

1. Start Chatting
2. Install a Model
3. I tried “Llama 3.2 1b Instruct”
4. Load the model
5. Start chatting

18.4 Using LLMs for Dynamic Research

What are the mechanics we need for this? We need our model to accept input and generate output and we need a method to transfer the elements of the conversation back and forth. The first stage is to think of the components we have used so far and how we might stitch them together for this task.

It is also a useful transitional step to work incrementally. First, maybe try to set up an interface to allow someone to talk to the model via a web interface. Then you might be able to figure out how to use GET and POST from PHP to communicate between two web servers. There are definitely better ways to do that, but it is a fairly direct starting point. If you want an additional challenge perhaps try to use the tool: curl.

 Classroom Exercise

Download and implement the above.
Start the GPT4All Server
See if you can get your GPT4All model to talk to another group's model.

18.5 Why Might You Want To Do This?

Durably reducing conspiracy beliefs through dialogues with AI

<https://www.youtube.com/watch?v=qD1fnYDTbHM>

This LinkedIn post talks about ChatGPT integration. What would you have to change to get it working with your local server?

 Class Question

What is a RAG in this context?

18.6 Next steps

Recently (Duan, Li, and Cai 2024) has published an R package to make all of this easier. The github repository has the library, a “read-me” and installation instructions. Our *class activity* is to see if anyone can get this up and working by next week.

Chapter 19

Databases and management

- What is it
- A book on DuckDB - database stuff
- MariaDB This SQL like, and open source. Might be easier to get started with and still be SQL enough to give them some professional benefits. I was thinking we could get some data online, often they come as CSV's and read it into the database? This is one example how. A blog that compares MariaDB to SQL. A quickie tutorial.
- Why would I want to use a relational database over a csv file (or R data frame or similar)? This could be a class exercise and discussion.
- *Exercise* Download MariaDB

19.0.1 Datascience

- one person's roadmap for non-cs grads

19.0.2 Grant Funding

At the moment I am not sure if will have time for this. But thinking of having the students review the peer review manual for NSERC Discovery Grants. Then have them each write a minimal proposal. Assign the proposals to members of the class, and then hold our own reviewers meeting to decide which projects to fund. Top grants get performed for experiments? Get extra-credit points?

Chapter 20

Summary

In summary, I have not gotten to writing a summary yet.

Appendix A

Mac Related Instructions

A.1 Homebrew

Homebrew installation instructions

A.2 Git

Install Mac

A.3 VS Code

Install Mac

A.4 Texlive

Get it via Homebrew with `brew install texlive` then export to pdf via quarto works well.

A.5 UV

This is a new tool that will take care of installing python version, python libraries, and initiating virtual environments. Many operating systems come with a version of python already installed. If you install additional versions to the global workspace you can get conflicts that can be very frustrating and challenging to fix.

First, get uv.

Second, install a version of python.

Third, practice setting up a virtual environment.

Fourth, install some packages.

A.6 R

Install instructions

A.7 Quarto

Getting started

Appendix B

Windows Specific Advice

B.1 Windows Subsystem for Linux

This is a relatively new feature for the Windows operating system that lets you install a linux virtual machine inside your current Windows distribution. You can thus “use” linux while still running windows. You can use it just for terminal commands or you can run some visual applications as well.

B.1.1 Make sure you are on Windows 10 or 11 and that you are updated.

B.1.2 Then follow the instructions per microsoft

B.2 VSCode

Install

B.3 Git

If you are using WSL then you probably already have it. Enter you WSL terminal and try `git --version`. If you see an answer you have it. If you don't want to use WSL you can install it for Windows directly.

B.4 Python

First, get uv.

Second, install a version of python.

Third, practice setting up a virtual environment.

Fourth, install some packages.

Appendix C

Python

Appendix D

Virtual Environments

The advantage of a virtual environment is the ability to isolate libraries you may install for a particular project from interfering with other projects or previously installed libraries. The disadvantage is that they do involve some additional steps and complexity and you may end up installing multiple versions of the same library, which can be an issue if hard disk space on your computer is limited.

D.1 Methods

There is more than one tool for creating a python virtual environment, but one of them is built into the standard python installation: *venv*, and so for simplicity this is the one that I recommend starting with.

D.1.1 Creating and Activating the Venv

If you are in a console you can simply run these commands in the terminal.

```
python -m venv <dir-where-you-want-venv>
cd <dir-where-you-want-venv>
source ./bin/activate
```

i Note

Depending on the specifics of your operating system and python installation you may find you need to use **python3** and **pip3** commands instead of plain **python** and **pip**.

If you are using a Windows device you will have to find your equivalent of a linux terminal. This can be using WSL or something like MinGW.

Once you have activated your `venv` you will see a change in the terminal display that indicates the `venv` is active. Now when you run things like `pip install <something>` it will install that library to a sandbox that is only accessible when using python from within that virtual environment. You should note that this notion of a sandbox or virtual environment is also available for some other programming languages.

To deactivate the environment you run ,

```
deactivate
```

from within the `venv`. Look for the change in the terminal display. Note how activating the environment shows up in the terminal prompt.

```
britt@arts-220486 ~ % source p390venv/bin/activate
(p390venv) britt@arts-220486 ~ % deactivate
britt@arts-220486 ~ %
```

D.1.2 The Terminal is All You Need

If you are a minimalist or doing something simple you can invoke the python interpreter in your `venv` and complete your task there with no other tools, and using only the libraries required for your particular, limited task. However, if you use some companion tooling things may get more complicated and trickier to debug.

D.2 Visual Studio Code

VS Code tries to make using a `venv` easier, but this also means there is some indirection that may make it harder to puzzle out errors. The basic work flow is to make sure that VS code is working in the folder you want your `venv` to be in. If not, use the **open folder** menu of VS Code to do so. Then, you can use the **Command Palette** (found under the *View* menu tab) to locate the **Python Create Environment** command. If you have never created an environment before it will do so. This can take some time so be patient and make sure this process completes. If there was a virtual environment previously constructed VS Code will probably be able to figure that out and ask you if you want to re-use it. Usually you will, but if not you can select to destroy it and create a new one. You will probably also need to affiliate a particular python version to association to your environment. VS Code can often make a reasonable suggestion and you can also use the command **Python Select Interpreter** if necessary.

After you have done that you will need to move into the affiliated terminal window of your VS Code environment to install the necessary libraries. You will need to be deliberate and make sure that you are in the correct terminal. VS Code may have several of these open for different purposes if you are working on a complicated project.

Note that while it may be convenient to use the VS Code terminals you do not have to. If you have a terminal program on your computer accessible to you outside of VS Code you can use VS Code as the editor and then move to your other terminal for activating and implementing and compiling the python project.

D.2.1 Quarto Complications

When you embed a python code block in your .qmd document you will want to compile it. Quarto is assembling a document. It will need more than just the libraries necessary for compiling the python code. It also needs the library for the textual and graphical representation of the output that you will insert into your document. Specifically, it needs the `jupyter` python library. This means that whatever the python is doing, even if no additional libraries are needed, you will have to have done a `pip install jupyter` in your venv *and* you will need to have the venv active for the location where the qmd is stored and compiled.

It is worth knowing that although VS Code offers the convenience of buttons to invoke the quarto process it is *not* necessary. You can invoke all the quarto commands through a terminal directly. For instance `quarto preview <file-name>.qmd` will try to compile your file and will also usually launch your browser so you can see the html result. It will then watch the source file and update the html whenever it detects you have changed the source. This can be quite convenient for editing.

Duan, Xufeng, Shixuan Li, and Zhenguang G. Cai. 2024. “MacBehaviour: An R Package for Behavioural Experimentation on Large Language Models.” *Behavior Research Methods* 57 (1). <https://doi.org/10.3758/s13428-024-02524-y>.

